Page 1 - What this project is, and how it was built

What does it do?

A pretend hospital called **Ashbrook General** has **25 members of staff**. The computer sends them emails. Some are **phishing** emails - fake emails that try to trick you into clicking a bad link. Some are normal, safe emails.

Each person does one of three things: **clicks** the email, **reports** it as spam, or **ignores** it.

A **digital twin** watches what each person does, and learns to guess what they will do next. "Digital twin" means a software copy of a real thing that learns by watching it.

Everything is invented. The hospital, the people and the emails are all fake. No email is ever sent to any real person.

What problem does it solve?

Phishing training is usually the same for everybody. This project asks a better question: **can we guess, for each single person, which kind of trick will fool them?** If we can, the training can be given to the people who need it most.

There is one strict rule in the project. Every pretend person has secret numbers, called `hidden_probs`, that say how likely they are to click. **The twin is never allowed to look at them.** It must learn only by watching what people actually do. The secret numbers are used at the very end, only to mark the twin's work - like a teacher's answer key.

How was it built, step by step?

The project was saved 14 times in Git. Each save was one step:

1. **First version** - the simulation ran only in the terminal (black screen, text only).
2. **The people** - created 25 staff across 5 areas of the hospital.
3. **The emails** - wrote 51 emails (26 fake, 25 real) based on the HIM 1055 course table of attacks.
4. **Tidy numbers** - gave every email a simple id number from 1 to 51.
5. **Big split** - the one huge file `app.py` was split into a "look" part and a "decide" part.
6. **Folders** - split the project into `frontend/`, `backend/` and `database/`, and built a real database.
7. **Phones** - made the screen work on a narrow window and a phone.
8. **Documents** - wrote `README.md` and `STRUCTURE.md` to explain the project.

What technologies are used, and why?

Technology	What it is	Why this project needs it
Python	A programming language	Every line of code is Python. One language for the website and the maths
Streamlit	A tool that turns Python into a web page	Gives a working website without writing any HTML or JavaScript
pandas	Works with tables of data	Counts and groups the results for every chart and table
Altair	Draws charts	Turns those tables into the bar charts on screen
scikit-learn	Machine learning	Trains models to compare against the simple twin
SQLite	A database kept in one file	Lets me ask questions like "all hard fake emails aimed at billing staff"

Which folders and files matter most?

File or folder	What it is
`app.py`	The website. All 8 tabs. 1,416 lines
`frontend/`	Colours and screen layout only
`backend/`	All the thinking: 18 files
`backend/engine.py`	Runs **one email delivery**. The most important file
`backend/twin.py`	The model itself, and all file saving
`backend/reader.py`	Decides if a person spots the fake email
`database/`	The database and its questions
`data/`	The emails, the people, and every answer ever given

Page 2 - How the frontend and backend work

What is the frontend?

The frontend is **everything you see and touch**: buttons, charts, text, tables.

In most websites the frontend is HTML and JavaScript. **Mine is not.** It is written in Python, and **Streamlit** turns it into a web page. I wrote no HTML or JavaScript at all.

My frontend files:

- `app.py` - the whole screen: the sidebar and all 8 tabs
- `frontend/theme.py` - the colours, and the words used for a result
- `frontend/responsive.py` - makes the screen work on a phone

What is the backend?

The backend is **everything that decides**: the rules, the maths and the data. It never draws anything.

My backend is the `backend/` folder - 18 files. The three that matter most:

- `backend/engine.py` - runs one email delivery from start to finish
- `backend/twin.py` - the model that learns and guesses
- `backend/reader.py` - looks at an email and decides if it is spam

One rule holds this together: no file in `backend/` may use Streamlit, so the backend does not even know a website exists. That is why the same simulation also runs from the terminal, using `main.py` or `live.py`.

What is an API, and how does it connect them?

An **API** is an agreed way for one piece of software to ask another for something - like a menu in a restaurant. You order from it without needing to know how the kitchen works.

In most websites the frontend and backend are **two separate programs** talking over the internet, using web addresses called **API endpoints**.

Mine is different, and say this clearly in the meeting: my frontend and backend are **one single program**, so there are **no API endpoints**. The frontend calls backend functions directly - like walking into the next room instead of posting a letter. For example, line 268 of `app.py`:

```
wf = workforce_frame(twin, people)
```

That asks `backend/metrics.py` for a table of results and gets it back at once.

Is there a database?

Yes - **SQLite**, a whole database kept in one single file, `data/simulation.db`, with no server to run.

- `database/schema.sql` - describes the 5 tables in plain SQL
- `database/build.py` - fills the database from the data files
- `database/db.py` - holds the questions, e.g. "all hard fake emails"

The tables are `emails`, `red_flags`, `people`, `hidden_probs` and `deliveries`. The secret `hidden_probs` sit in **their own table on purpose**, so using them by accident would be obvious.

The database is not the real truth. That is a plain text file, `data/history_log.jsonl`. Every answer is added to the end and never changed, and the database can be rebuilt from it at any time.

Simple diagram of what connects to what

```
   YOU -> web browser -> Streamlit      (the only internet part)
                          |
            +-------------+---------------+
            |                             |
       FRONTEND                       BACKEND
     app.py                    backend/engine.py  - runs one delivery
     frontend/theme.py           +- email_builder.py   make an email
     frontend/responsive.py      +- twin.py            guess, then learn
                                 +- reader.py          answer the email
                                 +- feedback.py        explain the result
                                        |
                                 database/db.py  ->  data/simulation.db
                                 data/history_log.jsonl   <- the real truth
```

Page 3 - What happens, step by step

The most important feature is the **Live tab**: emails arrive one at a time, and you watch the twin's guess change. Here is everything that happens, in order.

Step 1 - You start the program. You type `python3 -m streamlit run app.py`. Streamlit starts a small web server on your own computer and opens your browser. *Why:* this is the only way to see the project as a web page. *File:* `app.py`

Step 2 - The program loads its data. It reads the 25 people from `data/hospital_people.json`, and the 51 emails from the database. Then it reads all 11,544 past answers from `data/history_log.jsonl` and rebuilds what the twin has learned. *Why:* the twin must remember what it learned in earlier sessions. *Files:* `backend/persona.py`, `backend/generator.py`, `backend/twin.py`

Step 3 - The screen appears. The sidebar and all 8 tabs are drawn. *Why:* so you can choose what to do. *File:* `app.py`, helped by `frontend/theme.py`

Step 4 - You click "Start" on the Live tab. *Why:* this tells the program to begin delivering emails. *File:* `app.py` (line 525)

Step 5 - A new email is built. The program takes a real, safe hospital email and damages it: it changes the sender address, hides a bad link, and adds urgent words. Each change also records the clue it leaves behind. *Why:* a fake email built from a real one is much harder to spot, so the test is fair. *Files:* `backend/email_builder.py`, `backend/modifications.py`, `backend/entities.py`

Step 6 - The moment is decided. Is it day, evening or night? Is the person calm or very busy? *Why:* a tired, rushed person checks fewer things. *File:* `backend/situation.py`

Step 7 - The twin makes its guess. THIS IS THE KEY STEP. The twin looks only at its own counters for that person and divides two numbers to get a probability. It does this **before** anyone answers. *Why:* a guess only counts if it was made before the answer existed. *File:* `backend/twin.py`, called from `backend/engine.py` line 114

Step 8 - The person answers the email. The pretend person checks the email: Is the sender address right? Does the link go somewhere strange? Are there urgent, pushy words? If anything looks wrong, they report it as spam. If they are rushed, they only do the first 2 checks instead of all 8. *Why:* this copies how a real person reads email when busy. *File:* `backend/reader.py`

Step 9 - The twin learns, and the answer is saved. One counter goes up. The answer is written to the end of `data/history_log.jsonl` and pushed onto the disk straight away. *Why:* so nothing is lost if the computer crashes. *File:* `backend/twin.py`

Step 10 - The result appears on screen. The email is shown exactly as the person saw it, with a coloured label (CLICKED in red, RECOGNISED in blue), the counters, and a line chart showing the probability moving. *Why:* so you can watch the twin learn. *Files:* `app.py`, `frontend/theme.py`

Steps 5 to 9 then repeat, 12 times, before the screen refreshes.

The four steps that matter, in one picture

Everything above is really this small loop, run again and again. It lives in one file, `backend/engine.py`, so the Live tab and the Run tab can never disagree about how a delivery works.

```
1. PICK      build a new email from a real one       email_builder.py
      |
      v
2. GUESS     the twin says how likely a click is     twin.py
      |      <-- this MUST happen before step 3
      v
3. ANSWER    the person clicks, reports or ignores   reader.py
      |
      v
4. LEARN     the counter moves, the answer is saved  twin.py
      |
      +--> back to step 1
```

If step 2 and step 3 were swapped, the program would still run perfectly and report excellent results - and every one of those results would be worthless, because the twin would be copying the answer instead of predicting it. **That single ordering is the most important line of thinking in the project.**

Page 4 - How to explain this in the meeting

Your 3-5 minute explanation

> My project is called ALERT. A pretend hospital with 25 staff receives fake and real emails. Each person clicks, reports or ignores them. ALERT watches and learns to guess what each person will do next.
>
> The interesting rule is that every pretend person has secret click numbers, and the twin is never allowed to see them. It must learn only from what it observes. I keep those secrets in a separate database table, so using them by accident would be obvious. They are only used at the end, to mark the twin's work.
>
> Technically it is one Python program. Streamlit gives me a website with 8 tabs, `backend/` does all the thinking, and `database/` stores everything in SQLite. The rule that holds it together is that no backend file may use Streamlit, so the backend does not know the website exists - which is why the same simulation also runs from the terminal.
>
> The model itself is deliberately very simple: the chance of clicking equals the number of times they clicked, divided by the number of times they clicked plus reported. I then built a full machine learning model with 15 pieces of information and 200 decision trees, to see if it could do better. On emails it had never seen, the machine learning model scored 92.0% and my simple counting model scored 91.5%. Half a percent. I think that is a real finding, not a disappointment.

The three most important decisions

1. **The guess is made before the answer exists.** Three lines in `backend/engine.py`, in a fixed order. If I swapped two of them, the program would still run and report beautiful numbers that mean nothing.
2. **The backend never uses Streamlit.** This is why the project runs from the website, from the terminal, and from the test files.
3. **When testing the machine learning, I hold back whole emails, not random rows.** Each email was answered by about 225 people. A random split would put the same email in both the practice set and the test set, and the score would be a lie.

Questions your professor may ask

Why Streamlit and not React? It gave me a working website in Python only. My audience is a researcher, not a customer.

How do your frontend and backend talk? They are one program, so there is no internet connection between them. The frontend calls backend functions directly.

What are your API endpoints? There are none, and I would not invent any. The equivalent is a list of Python functions such as `twin.predict()`.

Why SQLite and not a big database? It is one file, needs no server, and comes free with Python. A few thousand rows for one user.

Where did your data come from? I generated it. The emails come from the HIM 1055 course table of attacks, so each one names the technique it shows.

Is fake data a weakness? Yes, and it is the main one. But it gives me a correct answer to check against, which real people never would.

How do you know the twin is learning? Its guess moves towards the real answer, and I test it on brand new emails it has never seen. It matches 90.7% of the time.

What was hardest? Keeping the test honest - making the guess first, never reading the secret numbers, and holding back whole emails.

How did you test it? Not well enough. The 4 files in `tests/` only print results for a human to read; they do not check themselves automatically.

Which parts did AI help with? I built it with Claude Code and kept `prompts.md` as a record. The typing was assisted; the decisions about how to test honestly are mine, and I can explain any file.

What could be improved

- **Write real automatic tests.** The biggest gap - nothing checks itself today.
- **List the required packages** in `pyproject.toml`, so others can install it.
- **Fix a bug I found** in `backend/benchmark.py`: it counts the correct action on a safe email as a mistake, making both scores look far worse than they are.
- **Use real people's answers.** The Inbox tab already collects them; even 500 would make every claim much stronger.
- **Let people improve** - the pretend people never learn from their feedback.